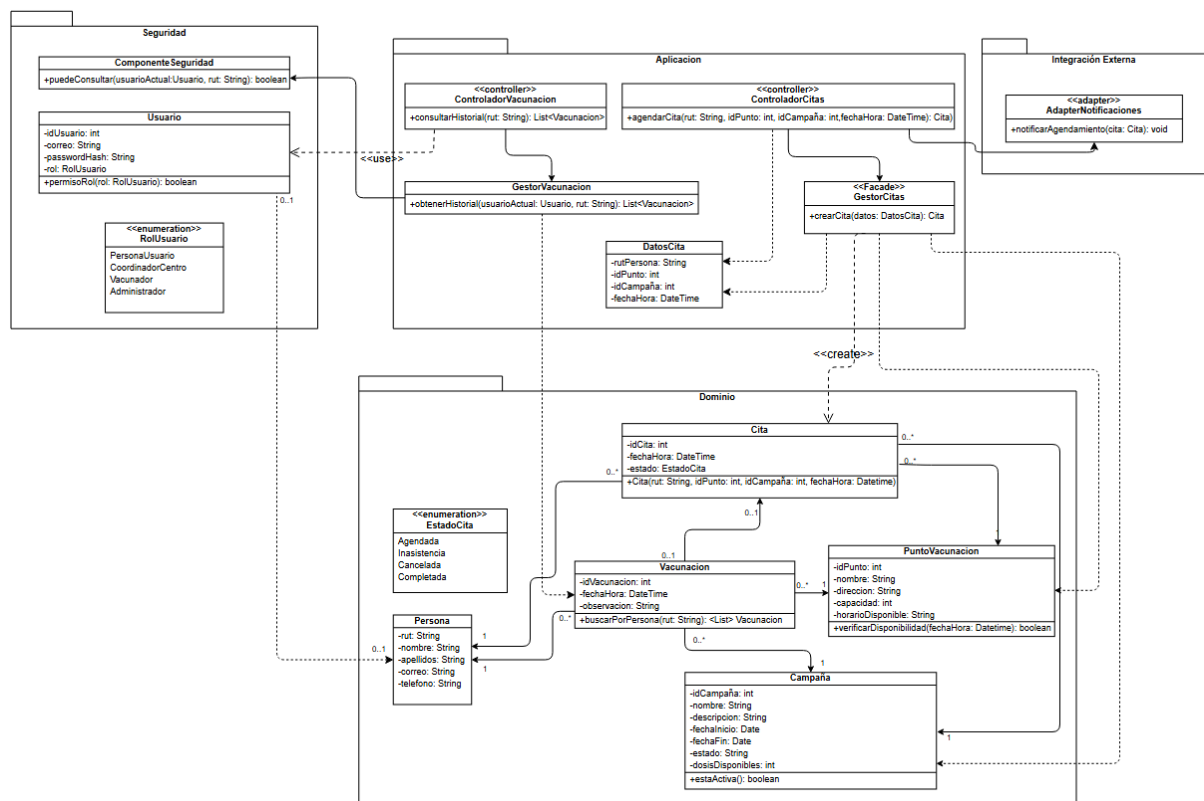


Diagrama de clases

El presente diagrama corresponde a una versión refinada del modelo de dominio desarrollado durante la Entrega 1. En esta nueva versión se incorporan métodos derivados de los mensajes presentes en los diagramas de comunicación y se explicita la navegabilidad entre las clases participantes.

De esta manera, el modelo no solo representa las entidades principales del sistema, sino también la forma en que los objetos colaboran durante la ejecución de los casos de uso seleccionados: **agendar una cita de vacunación** y **consultar el historial de vacunación de una persona**.

El diagrama se organiza en cuatro áreas: **seguridad**, **aplicación**, **dominio** e **integración externa**. Esta separación permite distinguir las responsabilidades propias de cada parte del sistema y evita concentrar toda la lógica en una sola clase.



Nota: Debido al tamaño del diagrama, la versión incorporada esta figura puede verse con una legibilidad reducida. Para evitar confusiones, se incluyó una copia con mayor resolución en el repositorio de GitHub, con el nombre UML_clases.png.

Organización Del Diagrama

El paquete **Dominio** contiene las entidades principales del sistema: Persona, Cita, Vacunacion, PuntoVacunacion y Campaña.

La clase Persona representa a una persona registrada en el sistema y almacena sus datos básicos. Una persona puede tener múltiples citas y recibir múltiples vacunaciones.

La clase Cita representa una atención programada dentro de una campaña y en un punto de vacunación determinado. Esta clase almacena su identificador, fecha, hora y estado. Además, incorpora un constructor que permite crear una cita a partir de los datos obtenidos durante el proceso de agendamiento.

La clase Vacunacion representa el registro de una dosis aplicada. Esta entidad almacena la fecha, la hora y las observaciones correspondientes. También incorpora el método buscarPorPersona(rut), utilizado para recuperar el historial de vacunación asociado a una persona determinada.

La clase PuntoVacunacion representa el lugar donde se realiza la atención. Debido a que conoce su capacidad y sus horarios disponibles, se le asigna la responsabilidad de verificar si existe disponibilidad para una fecha y hora determinadas mediante el método verificarDisponibilidad(fechaHora).

La clase Campaña representa una campaña de vacunación. Debido a que contiene sus fechas de inicio y término, su estado y sus dosis disponibles, se le asigna la responsabilidad de determinar si se encuentra vigente mediante el método estaActiva().

El paquete **Aplicación** contiene las clases responsables de coordinar los casos de uso. Se incorporan en este: ControladorCitas, GestorCitas, ControladorVacunacion, GestorVacunacion y DatosCita.

Los controladores reciben las solicitudes iniciales provenientes de los actores externos. Posteriormente, delegan la lógica del proceso a los gestores correspondientes.

La clase GestorCitas simplifica el proceso de agendar al momento coordinar las validaciones necesarias antes de crear una cita.

La clase DatosCita agrupa los datos requeridos para ejecutar esa operación: RUT de la persona, identificador del punto de vacunación, identificador de la campaña y fecha y hora seleccionadas.

El paquete **Seguridad** contiene las clases Usuario, RolUsuario y ComponenteSeguridad. La clase Usuario representa la cuenta autenticada y almacena un hash de la contraseña, evitando registrar directamente la credencial original. La enumeración RolUsuario permite distinguir los perfiles de persona usuaria, coordinador de centro, vacunador y administrador. Por su parte, ComponenteSeguridad centraliza la validación de permisos mediante el método puedeConsultar(usuarioActual, rut).

Finalmente, el área de **Integración externa** contiene AdapterNotificaciones. Esta clase representa el adaptador utilizado para solicitar el envío de confirmaciones mediante un servicio externo, evitando que el resto del sistema dependa de los detalles técnicos de esa integración.

Correspondencia Mensaje - Método

Los métodos incorporados al diagrama refinado se obtuvieron directamente a partir de los mensajes identificados en los diagramas de comunicación. En el caso del agendamiento de citas, el mensaje agendarCita(rut, idPunto, idCampaña, fechaHora) se transformó en el método agendarCita(...) de la clase ControladorCitas, mientras que el mensaje crearCita(datos) dio origen al método crearCita(...) de GestorCitas.

Las validaciones realizadas durante este proceso también se reflejan en el diagrama mediante los métodos verificarDisponibilidad(fechaHora) de la clase PuntoVacunacion y estaActiva() de la clase Campaña. Asimismo, la creación de una nueva cita se representa mediante el constructor de la clase Cita, y el envío de la confirmación se modela con el método notificarAgendamiento(cita) de AdapterNotificaciones.

Para el caso de uso de consulta del historial de vacunación, el mensaje consultarHistorial(rut) se incorporó como un método de ControladorVacunacion, mientras que obtenerHistorial(usuarioActual, rut) se agregó a la clase GestorVacunacion para coordinar la recuperación de la información.

La validación de permisos se representa mediante el método puedeConsultar(usuarioActual, rut) de ComponenteSeguridad, y la búsqueda de los registros asociados a una persona se modela mediante el método estático buscarPorPersona(rut) de la clase Vacunacion.

Los mensajes de retorno, como disponible, activa, citaCreada, autorizado, listaVacunaciones e historial, no se incorporan como métodos independientes, ya que corresponden a los resultados devueltos por las operaciones anteriores.